Software Design Document

In a system design interview, designing a notification system involves clarifying requirements, understanding constraints, and proposing a scalable, reliable architecture. Here's how you might approach it:

Step 1: Clarify Requirements

Functional Requirements

- Notifications should be sent via multiple channels (Email, SMS, Push).

- Support both real-time and scheduled notifications.

- Allow users to manage notification preferences.

- Support high volume dispatch (millions of notifications).

- Ensure retry mechanism for failed deliveries.

Non-functional Requirements

- High availability (24/7 service with minimal downtime).

- Scalability to handle variable loads efficiently.

- Low latency delivery for real-time notifications.

- Extensible to add new channels and features.

- Secure, ensuring data privacy and integrity.

Step 2: High-Level Design

Key Components

1. **Event Producer**: Sources of notification events, such as applications or system triggers.

2. **Notification Dispatcher**: Core component responsible for processing and managing notifications.

3. **Channel Handlers**: Specific components for different channels like Email, SMS, and Push.

4. **Preference Manager**: Manages user preferences and ensures notifications respect user settings.

5. **Template Manager**: Handles notification templates, ensuring scalability and consistency.

6. **Scheduler**: Manages scheduled notifications.

7. **Queue System**: Ensures decoupling and reliability.

8. **Database**: Stores user preferences, templates, and logs.

Step 3: Detailed Design

Workflow

1. **Event Trigger**: Events generated by applications are sent to a messaging queue.

2. **Notification Dispatcher**: Consumes messages from the queue, checks user preferences, fetches templates, and forwards to appropriate channel handlers.

3. **Channel Handlers**: Implement protocols for different channels and manage retries.

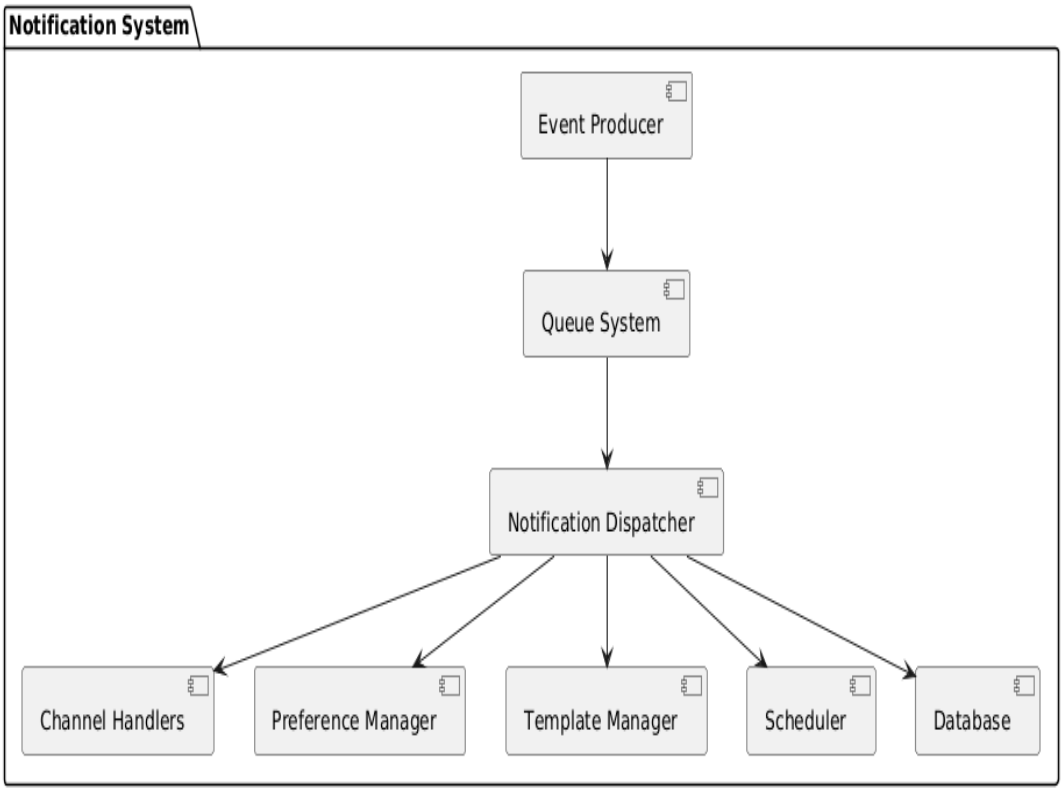
4. **Preference & Template Management**: Fetch preferences and templates from the database to construct final notifications.

5. **Scheduler**: Manages future notifications by scheduling them in the queue.

Step 4: Architecture Diagram

Here's a PlantUML diagram representing the high-level architecture of the notification system:

```plantuml
```




```

**Step 5: Considerations**

- **Scalability**: Use cloud-native solutions for auto-scaling, leveraging services like AWS SNS, SQS, or Kafka for messaging and Pub/Sub patterns.

- **Reliability**: Implement idempotency for retries, ensure at-least-once or exactly-once delivery semantics where needed.

- **Security**: Encrypt sensitive data, use HTTPS, and ensure proper access controls.

- **Monitoring and Logging**: Include comprehensive logging and metrics to monitor system health and performance.

This approach provides a robust, scalable foundation for a notification system that can grow and evolve with user needs and technological advancements.
```